STAT 432 Homework 11

Conner Jensen (connerj2)

Contents

Question 1: Implementing K-means **1**

(a) . . . 2

(b)–(c) . . . 3

Question 2: Image color compression **5**

(b) . . . 5

(c) . . . 7

Question 3: Hierarchical clustering of cars **7**

(a) . . . 7

(b) . . . 9

Question 1: Implementing K-means

I wrote my own K-means. It is not `stats::kmeans()`. Seed `432111`. The 10×2 data are i.i.d. $N(0, 1)$. $K = 2$.

```
kmeans_fit <- function(X, K, init, maxit = 100) {
  X <- as.matrix(X)
  n <- nrow(X)
  p <- ncol(X)
  C <- as.integer(init)
  means <- matrix(NA_real_, K, p)
  for (k in seq_len(K)) {
    means[k, ] <- colMeans(X[C == k, , drop = FALSE])
  }
  for (iter in seq_len(maxit)) {
    d2 <- outer(rowSums(X^2), rowSums(means^2), "+") -
      2 * X %*% t(means)
    Cnew <- max.col(-d2, ties.method = "first")
    for (k in seq_len(K)) {
      idx <- which(Cnew == k)
      if (length(idx) > 0L) {
        means[k, ] <- colMeans(X[idx, , drop = FALSE])
      }
    }
```

```
    if (identical(Cnew, C)) {
      C <- Cnew
      break
    }
    C <- Cnew
  }
  for (k in seq_len(K)) {
    idx <- which(C == k)
    if (length(idx) > 0L) {
      means[k, ] <- colMeans(X[idx, , drop = FALSE])
    }
  }
  wcss <- 0
  for (k in seq_len(K)) {
    idx <- which(C == k)
    if (length(idx) > 0L) {
      Xk <- X[idx, , drop = FALSE]
      wcss <- wcss + sum(sweep(Xk, 2, means[k, ])^2)
    }
  }
  list(cluster = C, means = means, wcss = wcss, iter = iter)
}

set.seed(432111)
X <- matrix(rnorm(10 * 2), 10, 2)
init_a <- sample(rep(1:2, length.out = 10))
init_b <- sample(rep(1:2, length.out = 10))
init_c <- sample(rep(1:2, length.out = 10))
```

Each start puts at least one point in each cluster. If a cluster goes empty later, that mean is left as it was. Means are recomputed from the final labels before the function returns.

(a)

One update: means of the initial groups, then nearest-mean labels.

```
m0 <- rbind(
  colMeans(X[init_a == 1, , drop = FALSE]),
  colMeans(X[init_a == 2, , drop = FALSE])
)
d2 <- outer(rowSums(X^2), rowSums(m0^2), "+") - 2 * X %*% t(m0)
C1 <- max.col(-d2, ties.method = "first")
round(m0, 3)
```

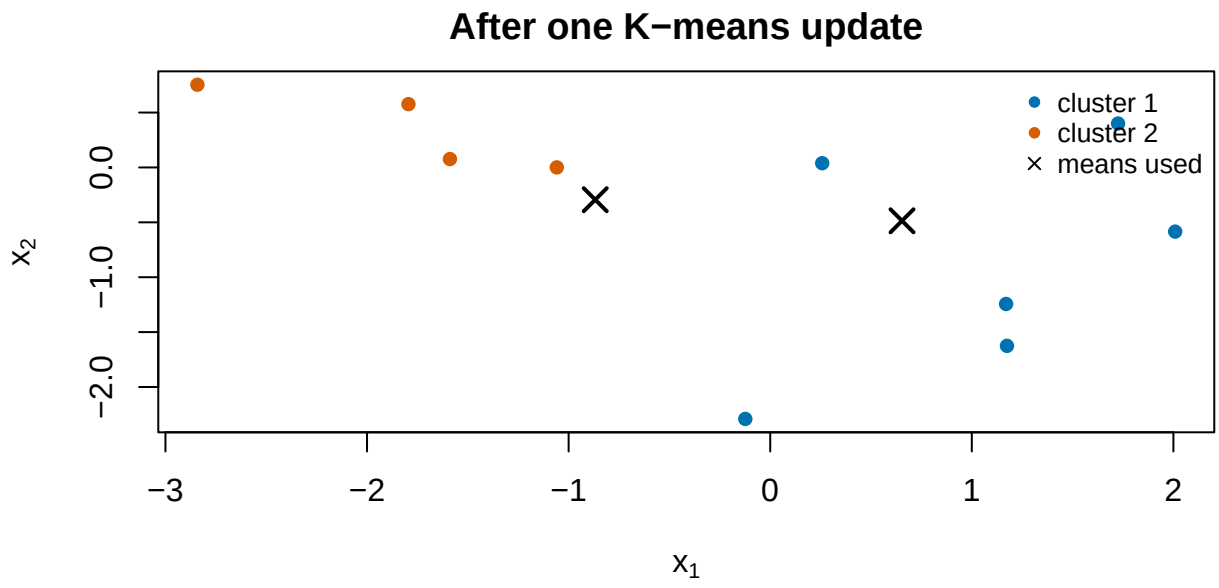
```
##        [,1]   [,2]
## [1,]  0.654 -0.486
## [2,] -0.868 -0.294
```

```
data.frame(obs = 1:10, init = init_a, after = C1)
```

```
##    obs init after
```

```
## 1    1    1      1
## 2    2    2      1
## 3    3    2      1
## 4    4    2      2
## 5    5    2      2
## 6    6    1      1
## 7    7    1      1
## 8    8    2      2
## 9    9    1      1
## 10  10    1      2
```

```r
cols <- c("#0072B2", "#D55E00")
par(mar = c(4.2, 4.2, 2.4, 0.8))
plot(X[, 1], X[, 2], pch = 16, col = cols[C1],
     xlab = expression(x[1]), ylab = expression(x[2]),
     main = "After one K-means update")
points(m0[, 1], m0[, 2], pch = 4, cex = 1.6, lwd = 2)
legend("topright", bty = "n", cex = 0.8,
       legend = c("cluster 1", "cluster 2", "means used"),
       col = c(cols, "black"), pch = c(16, 16, 4))
```



Points 2, 3, and 10 switched groups. The crosses are the means from the random start, not the means of the new labels.

(b)–(c)

```r
fit_b <- kmeans_fit(X, 2, init_b)
fit_c <- kmeans_fit(X, 2, init_c)
data.frame(
  start = c("B", "C"),
  wcss = round(c(fit_b$wcss, fit_c$wcss), 3),
```

```
  iter = c(fit_b$iter, fit_c$iter)
)
```

```
##   start   wcss iter
## 1     B 10.746    4
## 2     C 10.746    5
```

```
rbind(B = fit_b$cluster, C = fit_c$cluster)
```

```
##   [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## B    1    1    1    2    2    1    1    2    1     2
## C    1    1    1    2    2    1    1    2    1     2
```

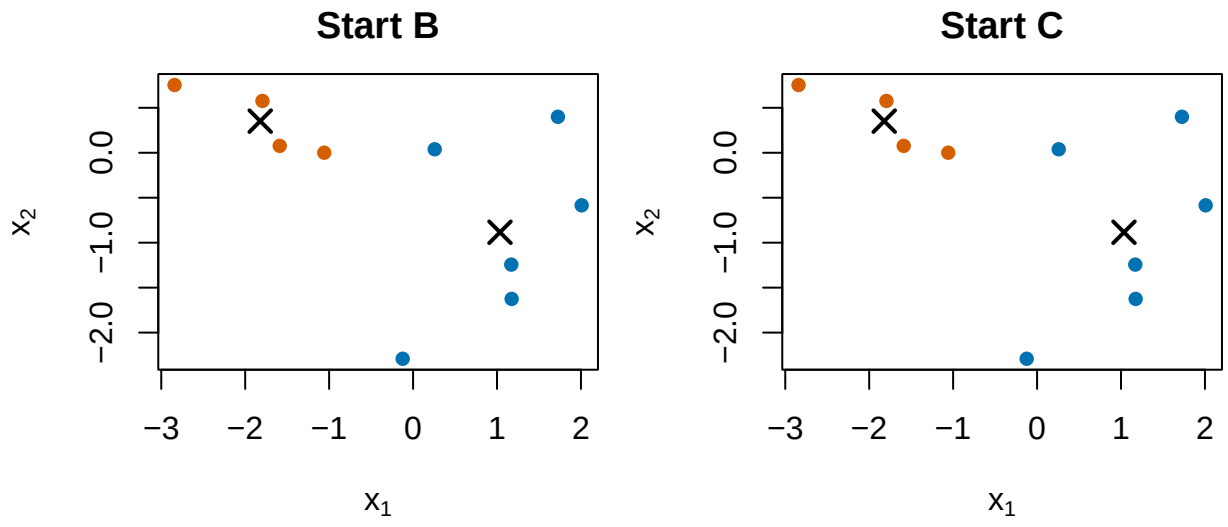
```
round(fit_b$means, 3)
```

```
##        [,1]   [,2]
## [1,]  1.035 -0.884
## [2,] -1.821  0.352
```

```
round(fit_c$means, 3)
```

```
##        [,1]   [,2]
## [1,]  1.035 -0.884
## [2,] -1.821  0.352
```

```
par(mfrow = c(1, 2), mar = c(4.2, 4.2, 2.6, 0.6))
plot(X[, 1], X[, 2], pch = 16, col = cols[fit_b$cluster],
     xlab = expression(x[1]), ylab = expression(x[2]),
     main = "Start B")
points(fit_b$means[, 1], fit_b$means[, 2], pch = 4, cex = 1.5, lwd = 2)
plot(X[, 1], X[, 2], pch = 16, col = cols[fit_c$cluster],
     xlab = expression(x[1]), ylab = expression(x[2]),
     main = "Start C")
points(fit_c$means[, 1], fit_c$means[, 2], pch = 4, cex = 1.5, lwd = 2)
```



The two runs find the same grouping and the same WCSS (10.75). The labels match too. On this draw both starts land in the same place. That does not mean K-means is unique in general; a worse start can stick at a higher WCSS. I did not see that here.

Question 2: Image color compression

I used a photo of a red-and-white tulip. The file is 2400×2400. I resized it to 200×200 by taking a regular grid of pixels, as allowed. RGB values stay in $[0, 1]$.

```
img0 <- readJPEG("data/tulip.jpg")
dim(img0)
```

```
## [1] 2400 2400    3
```

```
h0 <- dim(img0)[1]
w0 <- dim(img0)[2]
m <- 200
small <- img0[round(seq(1, h0, length.out = m)),
              round(seq(1, w0, length.out = m)), ]
pix <- cbind(as.vector(small[, , 1]),
             as.vector(small[, , 2]),
             as.vector(small[, , 3]))
dim(pix)
```

```
## [1] 40000     3
```

```
show_rgb(small, "Original (200 x 200)")
```



The pixel matrix is 40000×3.

(b)

```r
set.seed(432112)
Ks <- c(2, 3, 5, 10)
fits <- vector("list", length(Ks))
names(fits) <- paste0("K=", Ks)
for (i in seq_along(Ks)) {
  K <- Ks[i]
  init <- sample(rep(seq_len(K), length.out = nrow(pix)))
  fits[[i]] <- kmeans_fit(pix, K, init, maxit = 200)
}
data.frame(
  K = Ks,
  wcss = round(sapply(fits, `[[`, "wcss"), 1),
  iter = sapply(fits, `[[`, "iter")
)
```

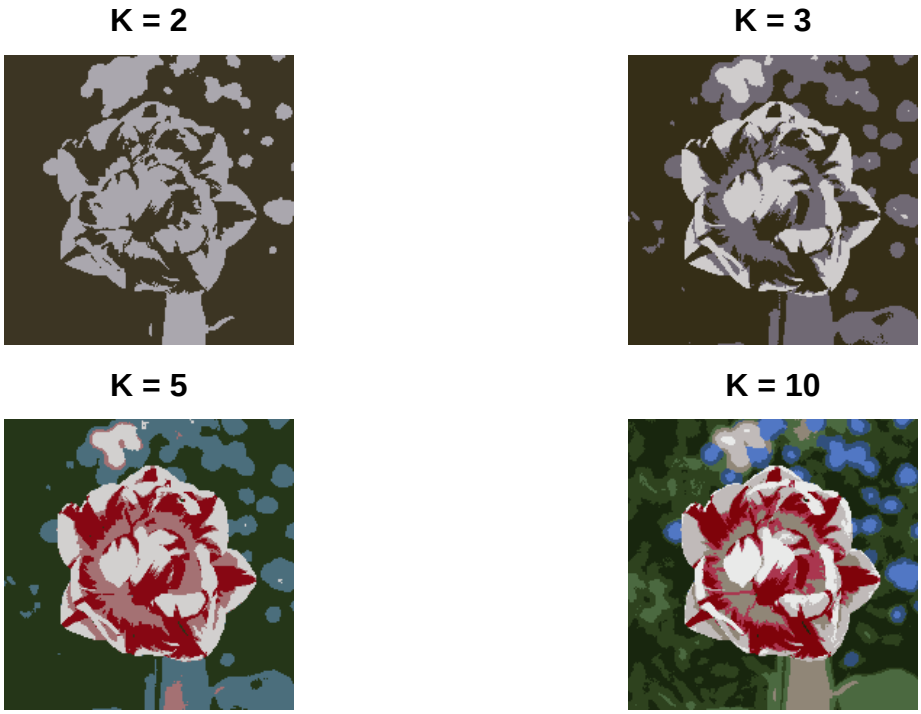
```
##       K   wcss iter
## K=2   2 3111.7   11
## K=3   3 2033.4   18
## K=5   5 1109.3   27
## K=10 10  457.4   91
```

```r
for (i in seq_along(Ks)) {
  cat("K =", Ks[i], "means (R, G, B)\n")
  print(round(fits[[i]]$means, 3))
}
```

```
## K = 2 means (R, G, B)
##       [,1]  [,2]  [,3]
## [1,] 0.235 0.208 0.137
## [2,] 0.668 0.654 0.684
## K = 3 means (R, G, B)
##       [,1]  [,2]  [,3]
## [1,] 0.812 0.800 0.800
## [2,] 0.206 0.180 0.090
## [3,] 0.437 0.413 0.456
## K = 5 means (R, G, B)
##       [,1]  [,2]  [,3]
## [1,] 0.296 0.431 0.486
## [2,] 0.642 0.445 0.450
## [3,] 0.530 0.029 0.075
## [4,] 0.145 0.212 0.094
## [5,] 0.822 0.815 0.813
## K = 10 means (R, G, B)
##        [,1]  [,2]  [,3]
##  [1,] 0.093 0.148 0.053
##  [2,] 0.497 0.011 0.039
##  [3,] 0.913 0.916 0.913
##  [4,] 0.665 0.212 0.309
##  [5,] 0.317 0.467 0.770
##  [6,] 0.565 0.524 0.466
##  [7,] 0.182 0.258 0.118
##  [8,] 0.752 0.728 0.724
```

```
##  [9,] 0.191 0.314 0.486
## [10,] 0.294 0.413 0.252
```

```r
par(mfrow = c(2, 2))
for (i in seq_along(Ks)) {
  show_rgb(rebuild(fits[[i]], m, m), paste("K =", Ks[i]))
}
```



WCSS falls as K grows: 3112, 2033, 1109, 457. Each extra color helps less than the last.

(c)

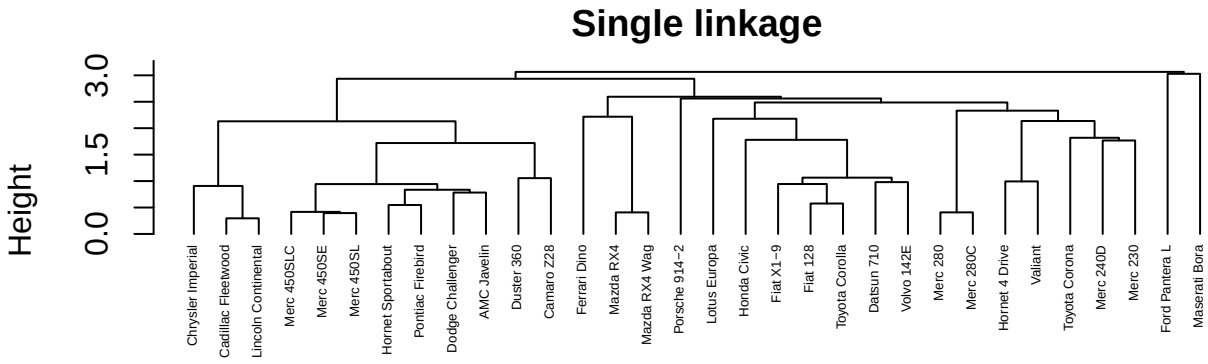
I would use $K = 5$. $K = 2$ is just light versus dark. $K = 3$ already separates the pale petals from the dark leaves, but the red in the flower is still muddy. $K = 5$ picks up a red cluster (mean about $(0.53, 0.03, 0.08)$) and the picture is readable. $K = 10$ has a much smaller WCSS, but I cannot see ten distinct colors, and I would rather keep a handful of palette colors.

Question 3: Hierarchical clustering of cars

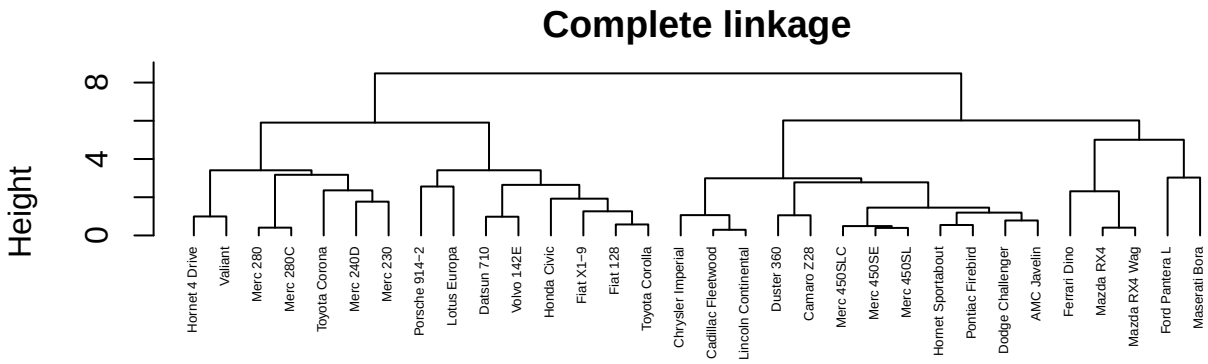
$n = 32$ cars, $p = 11$ numeric variables. I dropped `car` as a feature and used it as the label. Each column is scaled to mean 0 and SD 1. Distance is Euclidean.

(a)

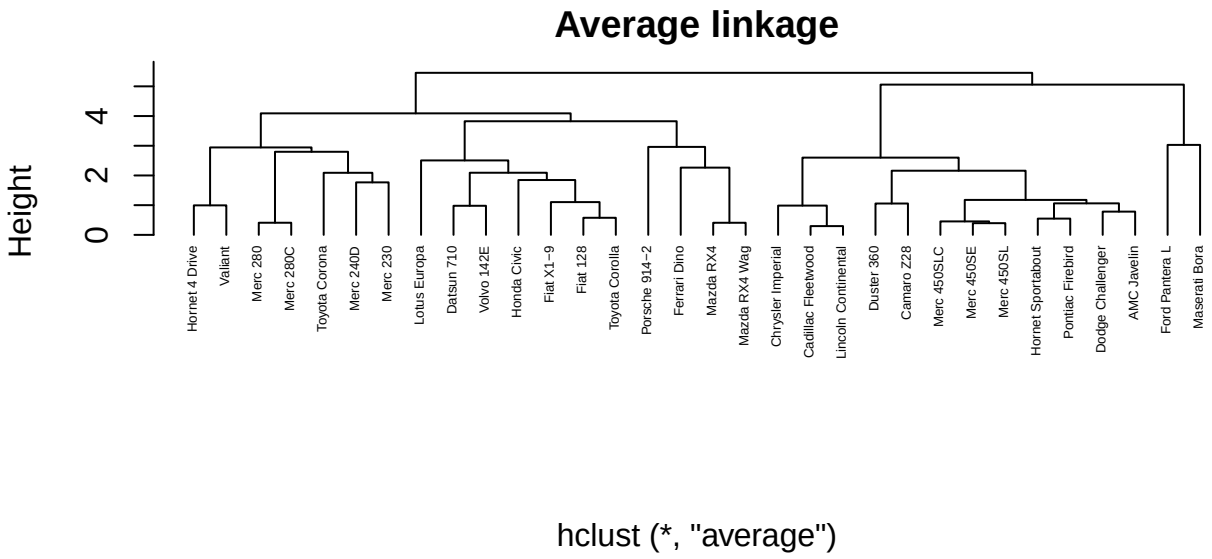
```r
cars <- read.csv("data/mtcars.csv")
lab <- cars$car
Xnum <- as.matrix(cars[, names(cars) != "car"])
Xs <- scale(Xnum)
rownames(Xs) <- lab
d <- dist(Xs, method = "euclidean")
hc_s <- hclust(d, method = "single")
hc_c <- hclust(d, method = "complete")
hc_a <- hclust(d, method = "average")
```



hclust (*, "single")



hclust (*, "complete")



Single linkage chains. Cut at $K = 3$ and you get one huge group plus two leftovers (`Ford Pantera L`, `Maserati Bora`). Complete and average stay more balanced. Complete splits the heavy V8s from the sporty manuals a bit more cleanly, so I use that one below.

(b)

I cut the complete-linkage tree at $K = 4$. The last few merge heights jump from about 3.4 to 5.0, so four groups is a reasonable place to stop.

```
cl <- cutree(hc_c, k = 4)
for (k in 1:4) {
  cat("Cluster", k, ":", paste(lab[cl == k], collapse = ", "), "\n")
}
```

```
## Cluster 1 : Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino, Maserati Bora
## Cluster 2 : Datsun 710, Fiat 128, Honda Civic, Toyota Corolla, Fiat X1-9, Porsche 914-2, Lotus Europa
## Cluster 3 : Hornet 4 Drive, Valiant, Merc 240D, Merc 230, Merc 280, Merc 280C, Toyota Corona
## Cluster 4 : Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL, Merc 450SLC, Cadillac Fleetwood,
```

```
tab <- aggregate(Xnum, by = list(cluster = cl), mean)
round(tab, 2)
```

```
##   cluster   mpg  cyl   disp     hp drat   wt  qsec   vs am gear carb
## 1       1 18.50 6.80 223.40 198.80 3.84 3.00 15.62 0.00  1 4.60 5.20
## 2       2 28.08 4.00  93.61  81.88 4.18 2.04 18.45 0.88  1 4.25 1.50
## 3       3 20.74 5.14 175.11 102.14 3.57 3.19 19.97 1.00  0 3.57 2.14
## 4       4 15.05 8.00 357.62 194.17 3.12 4.10 17.14 0.00  0 3.00 3.08
```

- Cluster 1: sporty manuals (RX4, Pantera, Ferrari Dino, Maserati Bora). High horsepower, `am = 1`, extra gears, short `qsec`.

- Cluster 2: light four-cylinders (Civic, Corolla, Fiats, Lotus, Porsche 914-2). Best mpg, smallest `disp` and `wt`.
- Cluster 3: midsize automatics (Hornet 4 Drive, Valiant, several Mercs, Corona). Straight-six-ish, `vs = 1`, `am = 0`.
- Cluster 4: heavy V8s (Cadillacs, Challenger, Camaro, Firebird, Duster, Merc 450s). Eight cylinders, large `disp`, worst mpg.

Standardizing matters because `disp` is in the hundreds and `am` is 0/1. Without scaling, Euclidean distance would mostly follow displacement and ignore the binary variables.